

---

# Experimental Report: Content-Aware Image Resizing via Seam Carving

---

Boya Wang  
2025133809

Code Link: [https://github.com/doddodod/26Spring\\_CS240\\_project\\_code\\_submission](https://github.com/doddodod/26Spring_CS240_project_code_submission)

## 1 Background and Related Work

### 1.1 Application Domain

Content-aware image resizing studies how to change the spatial size or aspect ratio of an image while preserving visually important content. This problem is common in photo editing, adaptive web design, mobile display adaptation, and general media retargeting systems. Traditional resizing operators can not preserve important content of the image. The target of content-aware resizing is to produce an image of a requested size while preserving salient objects, strong structures, and visually meaningful boundaries as much as possible.

In this project, the input is an image  $I$  of size  $m \times n$  and a target size  $m' \times n'$ . The output is a retargeted image  $I'$  whose dimensions match the target while reducing perceptual distortion. The practical setting is especially suitable for algorithmic reproduction because the visual problem can be expressed as a discrete optimization problem over pixels, paths, and local costs, and the resulting algorithm can be evaluated both qualitatively and by runtime/scalability experiments.

### 1.2 Algorithmic Formulations

The basic formulation assigns each pixel an energy value  $e(i, j)$  representing its visual importance. In the classical backward-energy method from Avidan and Shamir [2007], this energy is usually derived from image gradients, so pixels on strong edges or textured regions are expensive to remove. A vertical seam is an 8-connected path  $s = \{(x(i), i)\}_{i=1}^m$  containing exactly one pixel in each row and satisfying  $|x(i) - x(i-1)| \leq 1$ . The optimal seam minimizes the cumulative energy

$$s^* = \arg \min_s \sum_{(i,j) \in s} e(i, j). \quad (1)$$

For a 2D image, this is naturally solved by dynamic programming. If  $M(i, j)$  is the minimum cumulative cost of a seam ending at pixel  $(i, j)$ , then the backward-energy recurrence is

$$M(i, j) = e(i, j) + \min\{M(i-1, j-1), M(i-1, j), M(i-1, j+1)\}. \quad (2)$$

After filling the table row by row, the minimum entry in the last row gives the end of the optimal seam, and backtracking recovers the path. Horizontal seams are handled symmetrically. The main reproduction in this project follows this dynamic-programming view for both backward and forward energy.

Forward energy modifies the objective by estimating the disruption introduced after a pixel is removed. Instead of only asking which existing pixels have low energy, it asks which seam will create the smallest new neighbor differences in the retargeted image. For 2D images, the same dynamic-programming structure can be used for forward energy implementation after adding transition-dependent costs Rubinstein et al. [2008].

The forward seam carving proposed by Rubinstein et al. [2008] is a min-cut formulation for both 2D images and 2D seam manifolds in 3D video volumes. This project chooses the equivalent dynamic-programming implementation for the 2D image version of it. A further formulation treats retargeting as a sequence of different operators, such as seam carving, cropping, and scaling. Rubinstein et al. [2009] defines a resizing space and search for a good operator path using dynamic programming and an image similarity objective. In summary, the related work combines pixel-level seam DP, graph-cut seam surfaces, and DP search over operator sequences.

### 1.3 Representative Papers

**Seam carving for content-aware image resizing.** Avidan and Shamir introduced seam carving as a discrete image operator for content-aware reduction and enlargement Avidan and Shamir [2007]. The problem it addresses is the failure of scaling and cropping to adapt images to new aspect ratios while preserving important content. Its key algorithmic idea is to define low-energy connected seams and remove or insert them iteratively. This paper directly inspires the backward-energy implementation in this project: the core method is a shortest-path-like dynamic program over the image grid. The reported effect is that low-importance background pixels are removed before salient objects and strong structures, making the result often more natural than uniform scaling or cropping.

**Improved seam carving for video retargeting.** Rubinstein, Shamir, and Avidan identified two limitations of classical seam carving: backward energy can introduce new visual artifacts, and frame-by-frame image resizing creates temporal jitter in videos Rubinstein et al. [2008]. The paper contributes forward energy, which evaluates the cost introduced into the image or video after removing a seam, and a graph-cut formulation for finding valid seam manifolds in video. In this project, the main inspiration is the forward-energy criterion rather than the full video pipeline. I reproduce the 2D image version with dynamic programming, which is consistent with the paper’s explanation that forward energy can be encoded by DP for images. Its effect is a reduction of artifacts such as broken lines, inconsistent edges, and unnatural local discontinuities.

**Multi-operator media retargeting.** The multi-operator Rubinstein et al. [2009] proposes a resizing space of retargeting operators such as seam carving, cropping, and scaling, and it searches for an operator sequence that optimizes an image-to-image similarity measure. More concretely, if the target requires reducing  $N$  pixels and there are  $k$  available operators, then there is an exponential search space of roughly  $k^N$  possible operation sequences. The paper avoids this brute-force enumeration by using dynamic programming over the resizing-space lattice with a cost called Bi-Directional Warping (BDW), which evaluates how well the retargeted image preserves the source image under order-preserving row or column alignments. This paper is not directly implemented in my reproduction, but it shows how dynamic programming can be further used.

## 2 Chosen Paper / Method Analysis

### 2.1 Problem Formulation

This section analyzes the two methods reproduced in the implementation: backward-energy seam carving from Avidan and Shamir Avidan and Shamir [2007], and forward-energy seam carving from Rubinstein et al. Rubinstein et al. [2008]. Both methods take an image  $I$  and a target size as input, and both resize the image by repeatedly removing or inserting seams. For width reduction, a vertical seam removes one pixel from each row; for height reduction, the analogous horizontal seam removes one pixel from each column.

In backward-energy seam carving, the objective is to remove the seam with the smallest sum of existing pixel energies. If  $e(i, j)$  is the image-gradient energy at pixel  $(i, j)$ , the method assumes that low-gradient regions are less important and can be removed with less visible damage. The chosen seam is therefore the minimum-energy connected path through the image.

Forward-energy seam carving keeps the same seam constraint but changes the objective. It observes that a seam with low current energy can still introduce a large artifact after removal because pixels that were not neighbors before may become adjacent. The forward formulation estimates this newly introduced cost for each possible local seam transition and chooses the seam that minimizes the sum of current pixel energy and introduced neighbor differences.

## 2.2 Algorithm Description

**Backward energy.** The implementation first converts the image to grayscale and computes a Sobel-based gradient energy. For each vertical seam, it builds a cumulative cost table  $M$  and a parent table. The first row initializes the DP. For every later pixel  $(i, j)$ , the algorithm chooses the cheapest predecessor from the three valid pixels above it:

$$M(i, j) = e(i, j) + \min_{k \in \{j-1, j, j+1\}} M(i-1, k). \quad (3)$$

The algorithm then starts from the minimum entry in the final row and follows the parent pointers upward to recover the seam. Removing the seam shifts pixels after the seam one position left in each row. Seam insertion is implemented by first recording low-energy seams on a temporary carved image, then reinserting them into the original image with interpolated pixel values.

**Forward energy.** The forward-energy implementation uses the same DP table/backtracking structure, but each transition has an additional cost. When removing pixel  $(i, j)$ , the left and right neighbors in the current row become adjacent, contributing a middle cost. If the seam comes from up-left or up-right, an additional vertical neighbor difference is introduced. In simplified notation, for the three incoming directions the recurrence is

$$M(i, j) = e(i, j) + \min \begin{cases} M(i-1, j-1) + C_L(i, j), \\ M(i-1, j) + C_U(i, j), \\ M(i-1, j+1) + C_R(i, j), \end{cases} \quad (4)$$

where  $C_L$ ,  $C_U$ , and  $C_R$  are the local forward costs for left, up, and right transitions. Let  $g(i, j)$  be the grayscale intensity and use clamped boundary indices at the image border. Removing pixel  $(i, j)$  makes the left and right neighbors adjacent, giving

$$C_U(i, j) = |g(i, j+1) - g(i, j-1)|. \quad (5)$$

If the seam bends from the upper-left or upper-right predecessor, one additional new edge is introduced:

$$C_L(i, j) = C_U(i, j) + |g(i-1, j) - g(i, j-1)|, \quad (6)$$

$$C_R(i, j) = C_U(i, j) + |g(i-1, j) - g(i, j+1)|. \quad (7)$$

Illegal boundary bends are assigned infinite cost. This DP is sufficient for the 2D image formulation because the seam remains a one-dimensional connected path; for 3D video retargeting, the paper instead formulates the seam manifold as a network-flow/min-cut problem on a space-time graph.

## 2.3 Complexity Analysis

For an image of height  $m$  and width  $n$ , computing one backward-energy seam requires  $O(mn)$  time to compute the energy map and fill the DP table, and  $O(mn)$  memory for the cumulative cost and parent tables. Backtracking adds only  $O(m)$  time for a vertical seam. Removing one seam copies the image into an array of width  $n-1$ , also costing  $O(mn)$  in a straightforward implementation. Thus, reducing the width by  $r$  seams costs approximately  $O(rmn)$ .

Forward-energy seam carving has the same asymptotic complexity for 2D images. The only extra component is the transition-cost computation. Before each seam is found, the current image is used to compute three arrays,  $C_L$ ,  $C_U$ , and  $C_R$ , which estimate the new neighbor differences created by a left, vertical, or right seam transition. These arrays must be recomputed after every seam removal because the image width and pixel adjacencies have changed. This step costs  $O(mn)$  time and  $O(mn)$  memory for the current image, after which the DP still evaluates only three predecessors per pixel. Therefore, one forward-energy seam costs

$$O(mn)_{\text{energy}} + O(mn)_{\text{transition}} + O(mn)_{\text{DP}} + O(mn)_{\text{removal}} = O(mn).$$

Removing  $r$  seams remains  $O(rmn)$ , the same as backward-energy seam carving in Big-O notation. The practical difference is a larger constant factor caused by the transition-cost arrays and the transition-dependent DP update, which matches the empirical observation that forward energy is slower but follows the same scaling trend.

### 3 Reproduction and Implementation

This work uses the open-source implementation of Avidan and Shamir (2007) and Rubinstein et al. (2008) by Campbell (<https://github.com/andrewdcampbell/seam-carving>) as a reference. I reproduce the core algorithms with PIL and NumPy (handwritten Sobel, explicit DP), run the same demo images, and compare my outputs against its results to verify correctness. On this baseline, three DP-level optimizations on the backward-energy pipeline are evaluated.

#### 3.1 Implementation Details

The reproduction was implemented in Python in the project code directory. NumPy is used for array computation and dynamic programming, PIL is used for image I/O and the ordinary resizing baseline, and Matplotlib is used only for plotting experiment results. The implementation does not use a third-party seam-carving library; the energy computation, cumulative-cost DP, parent-pointer backtracking, seam removal, and seam insertion operations are implemented directly.

Images are represented as  $H \times W \times 3$  NumPy arrays with `float64` RGB values. A vertical seam is represented as a length- $H$  integer vector, where each entry stores the selected column index for one row. The dynamic program stores a cumulative cost table and a parent table, both of size  $H \times W$ . Optional masks are loaded as grayscale images and then thresholded into binary regions: pixels above the threshold are treated as protected or removable regions, depending on the operation.

For backward energy, the implementation converts the image to grayscale and computes a Sobel-based gradient energy map. The minimum-energy seam is found by filling the DP table row by row and then backtracking from the minimum value in the final row. For forward energy, the implementation keeps the same DP and backtracking structure but adds the transition costs introduced by removing the current pixel. Thus, the forward method changes the local objective while preserving the 2D dynamic-programming formulation.

Width reduction is implemented by deleting one vertical seam at a time. Height reduction is handled by transposing the image, applying the same vertical-seam procedure, and transposing the result back. Full 2D resizing is therefore performed by applying width and height changes sequentially. The implementation also supports seam insertion for enlargement: low-energy seams are first found and recorded on a temporary carved image, then inserted back into the original image with interpolated pixel values. For masks, grayscale values above the threshold 10.0 are treated as active mask pixels. Protective masks assign these pixels a large positive energy,  $10^6$ , so seams avoid marked content, while removal masks assign them a large negative energy,  $-10^8$ , so seams are attracted to the marked object. Object removal is performed by repeatedly carving seams through the removal mask and then inserting seams to restore the original image size.

#### 3.2 Reproduction of Reported Results

The main reproduction compares the original image with ordinary scaling, center cropping, backward-energy seam carving, and forward-energy seam carving. These comparisons reproduce the central qualitative behavior of seam carving: instead of uniformly distorting the whole image or removing a rectangular boundary region, the algorithm removes connected low-energy paths from less important areas. The implementation also reproduces the expected distinction between backward and forward energy. Backward energy chooses seams according to the current pixel energy, while forward energy penalizes seams that would introduce strong new neighbor differences after removal. In structured examples, this usually makes forward energy less prone to broken lines and local discontinuity artifacts. The comparisons are shown in Figure 6.

Several application-level effects from the original seam-carving paper are also reproduced qualitatively. Vertical and horizontal seam removal change image dimensions in one direction, seam insertion performs content-aware enlargement, protective masks preserve marked regions during resizing, and removal masks support simple object removal followed by seam insertion. The reproduction does not use a perceptual image-quality metric such as SSIM in this section; the quantitative evaluation is mainly runtime and scalability analysis, which is reported later in the experimental evaluation section. Visual examples of these reproduced effects are provided in Appendix A.

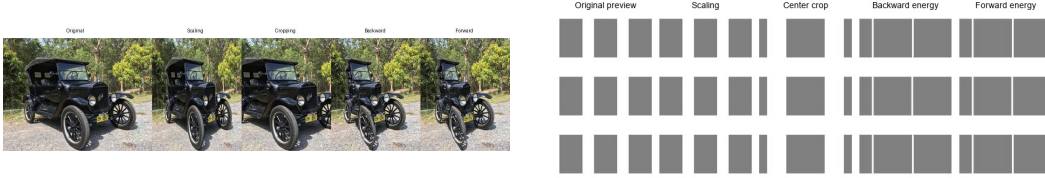


Figure 1: Failure-case examples for aggressive width reduction. The car example is resized from  $500 \times 375$  to  $300 \times 375$ , and the grid example is resized from  $256 \times 256$  to  $154 \times 256$ . Each panel compares the original preview image, ordinary scaling, center cropping, backward-energy seam carving, and forward-energy seam carving.

### 3.3 Deviations and Failures

This reproduction differs from the original papers in several ways. First, the backward-energy implementation uses a Sobel-gradient energy as the main importance function, while the original seam-carving paper discusses multiple energy functions and saliency cues. Second, the forward-energy implementation is restricted to 2D images. The improved seam-carving paper mainly targets video retargeting, where the seam becomes a 2D manifold in a 3D space-time volume and is solved using graph cuts; this project reproduces the image-based DP version of the forward-energy criterion instead. Third, the multi-operator paper is used as related work and motivation, but its BDW similarity measure and multi-operator optimization are not implemented.

Therefore, the results should be interpreted as representative reproduction results rather than pixel-identical recreations of the original paper figures. The project uses available demo images and local target sizes rather than the full original figure set, video data, and parameter choices.

The main failure cases are consistent with the limitations of seam carving as shown in Figure 1. For the left case, if an image is dense and contains little low-energy background, the algorithm may have no visually safe seam to remove and can cut through meaningful content. In the right case, images with long straight lines, regular grids, or repeated textures can still show bending artifacts, especially under aggressive resizing. Forward energy reduces some of these artifacts but does not remove the problem completely. However, the scaling operator protects the original features well, this highlights the optimal combination of operators as mentioned in the paper 3.

## 4 Experimental Evaluation

### 4.1 Experimental Setup

The main scalability experiment evaluates the runtime of the two reproduced seam carving algorithms: backward-energy seam carving and forward-energy seam carving. These two methods are treated as the algorithmic variants under comparison. To isolate the core vertical seam-removal loop, each input is resized from  $W \times H$  to  $(W/2) \times H$ , so the height is unchanged and the width is reduced by repeatedly removing vertical seams. This avoids mixing the measurement with the transposed horizontal-resizing stage.

To create a controlled scalability test, the experiment starts from one source image and first resamples it into six test inputs with the same aspect ratio but increasing pixel counts, from about  $6.7 \times 10^3$  pixels to about  $1.75 \times 10^5$  pixels. These prepared inputs are used only to vary problem size while keeping image content comparable. For each prepared input of size  $W \times H$ , the seam-carving task is then identical: reduce the width to  $W/2$  while keeping the height  $H$  unchanged. Thus, as the test resolution increases, both the cost of one DP pass and the number of seams to remove increase, making the experiment suitable for measuring scalability. Each method is repeated five times. The recorded metrics are the average runtime and standard deviation for backward energy and forward energy at each image size.

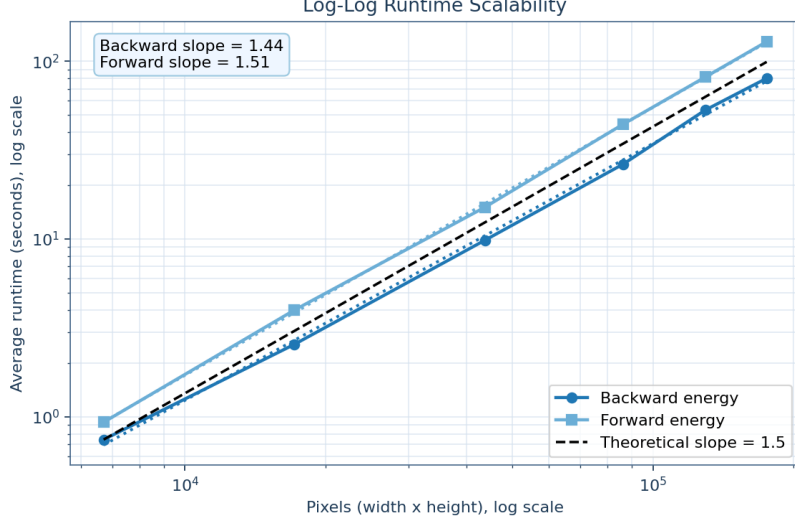


Figure 2: Log-log runtime scalability for backward-energy and forward-energy seam carving. Width is reduced by half at each input resolution. The empirical slopes are close to the expected  $O(N^{3/2})$  trend for repeated vertical seam removal.

## 4.2 Runtime and Scalability Analysis

The runtime trend is visualized with a log-log plot rather than the ordinary linear-coordinate plot. In linear coordinates, the largest image sizes dominate the visual scale and make the growth pattern at smaller sizes hard to inspect. For seam carving, the more important question is the power-law relationship between input size and running time, for example whether the implementation behaves like  $N^{1.5}$  for pixel count  $N$ . In log-log coordinates, a power-law relationship becomes approximately linear, and the fitted slope directly corresponds to the empirical complexity exponent. Therefore Figure 2 uses the log-log runtime plot to provide a more careful scalability analysis.

The measured runtimes increase rapidly with image size. Backward energy grows from about 0.74 seconds at the smallest tested resolution to about 80.29 seconds at the largest one. Forward energy grows from about 0.93 seconds to about 129.07 seconds over the same range. Forward energy is consistently slower, by roughly  $1.25\times$  to  $1.68\times$ , because each DP transition also evaluates the additional forward-energy costs. However, the two methods have the same asymptotic structure: both repeatedly compute a seam over the full current image and remove one seam at a time.

The log-log fitted slopes are approximately 1.44 for backward energy and 1.51 for forward energy. This is close to the expected  $O(N^{3/2})$  behavior for the width-halving experiment. The reason is that each vertical seam costs  $O(WH)$  work, and reducing the width by half removes  $\Theta(W)$  seams. Since the test images have approximately fixed aspect ratio,  $N = WH = \Theta(W^2)$ , so the total work scales as  $O(W \cdot WH) = O(W^3) = O(N^{3/2})$ . The main bottleneck is therefore the repeated full-image dynamic program. In this pure Python implementation, the nested per-pixel loops and repeated seam removals dominate runtime; forward energy adds a larger constant factor through its transition cost arrays.

## 4.3 Runtime Baselines and Complexity

Ordinary scaling and center cropping are used as qualitative baselines for image retargeting, but their runtime behavior is much simpler than seam carving. Center cropping only selects a rectangular window; aside from writing or copying the output image, the operation is essentially  $O(m'n')$  for a target image of size  $m' \times n'$ . Ordinary image scaling computes each output pixel by a fixed-size interpolation kernel, so its time complexity is also  $O(m'n')$ .

In contrast, backward-energy and forward-energy seam carving are optimization procedures. Removing one seam from an  $m \times n$  image costs  $O(mn)$ , and removing  $r$  seams costs approximately

$O(rmn)$ . Forward energy has the same asymptotic complexity as backward energy but a larger constant factor because it computes and applies transition-dependent costs. Thus, scaling and cropping are expected to be far faster as runtime baselines, while seam carving provides the content-aware behavior at the cost of substantially higher computation.

## 5 Optional Extension (Bonus-Level)

All the extensions are based on the forward-energy seam carving implementation. The baseline implementation removes seams one at a time. For one vertical seam, the pipeline consists of energy computation, DP initialization, DP table filling, endpoint selection, seam backtracking, and image deletion:

$$O(HW)_{\text{energy}} + O(W)_{\text{init}} + O(HW)_{\text{DP fill}} + O(W)_{\text{endpoint}} + O(H)_{\text{backtrack}} + O(HW)_{\text{remove}} = O(HW).$$

As bonus-level extensions, I implemented three acceleration strategies that target different parts of this pipeline. Batch seam extraction reduces repeated energy computation and image deletion by removing multiple seams together. Local DP update targets the DP table-filling stage by recomputing only an affected region. Multi-scale seam carving targets the seam search stage by finding a coarse seam at lower resolution and refining it in a narrow full-resolution band.

### 5.1 Further Optimizations and Heuristic Improvements

**Batch seam extraction.** The first extension reduces the number of energy recomputations and deletion passes by removing multiple seams in one batch. For the current image, the algorithm computes one full energy map and copies it into a working map. It then extracts several non-overlapping seams from this same working map before performing deletion. After one seam is found by a full DP pass, the pixels on that seam are set to  $\infty$  so that no later seam in the same batch can reuse the same pixels. In addition, a penalty band with default radius 2 is added around the seam, using a large penalty weight ( $10^6$ ), so that subsequent seams are encouraged to stay away from the already selected path. After  $b$  seams are collected, a boolean keep mask removes all selected seam pixels at once.

For an image of height  $H$  and width  $W$ , removing  $r$  seams with the baseline requires  $r$  rounds of energy computation, DP seam search, and image deletion. Each round is  $O(HW)$ , so the total cost is about  $O(rHW)$ . With batch size  $b$ , the algorithm still performs one DP seam search for each selected seam, giving  $r \cdot O(HW)$  DP work. The difference is in the update schedule: energy is computed only once per batch and image deletion is also performed once per batch, so these two operations occur  $\lceil r/b \rceil$  times instead of  $r$  times. Therefore the asymptotic order remains  $O(rHW)$ , and the main saving comes from reducing the time spent on repeated image deletion/copying and energy recomputation.

**Local DP update.** The second extension targets the DP table-filling step. In the baseline, after each seam removal the algorithm recomputes the full cumulative-cost table  $M(i, j)$  and the full parent table over the entire current image. The local-DP version keeps these tables from the previous round. After a seam is removed, it deletes the same seam columns from the old  $M$  table, shifts the stored parent indices into the new coordinate frame, and then recomputes only a pyramid-shaped affected region instead of the whole table.

This reuse is possible because the backward-energy function used here is local. It is based on a Sobel gradient, so removing one vertical seam changes pixel adjacencies only near the deleted path. Far from the seam, the local neighborhoods are unchanged after column alignment, so the energy values and the DP values whose dependency chains do not touch the changed neighborhood can be reused. In other words, the old DP table is not reused blindly: cells that may depend on changed local energy are marked as affected and recomputed with the same recurrence as the baseline.

The affected region is controlled by the hyperparameter  $\Delta$ . For row  $i$ , the algorithm starts from a band around the previous seam location,  $[s_i - \Delta, s_i + \Delta]$ . Since each DP cell depends on three cells in the previous row, this band is expanded downward to form a conservative pyramid region. A larger  $\Delta$  recomputes more cells and is safer for images whose energy changes propagate farther, while a smaller  $\Delta$  can be faster but is less conservative. Thus the best value depends on the local energy structure of the image.

For one seam, the baseline fills  $O(HW)$  DP cells. If the affected pyramid has average width  $B$ , the local update recomputes about  $O(HB)$  DP cells and copies the aligned old values elsewhere. For

removing  $r$  seams, the DP-filling part is therefore reduced from about  $O(rHW)$  to about  $O(rHB)$  when  $B \ll W$ . This does not eliminate the other parts of the pipeline: the implementation still computes the energy map and removes the seam from the image. The optimization is therefore specifically a reduction in repeated DP table filling, and the correctness test checks that the resulting image matches the full-DP baseline in the tested cases.

**Multi-scale seam carving.** The third extension uses a coarse-to-fine approximation. The image is first downsampled by a scale factor  $s$ ; in the implementation the default is  $s = 2$ , using simple stride decimation. A low-resolution seam is found on this smaller image. It is then mapped back to full resolution by assigning high-row  $i$  to low-row  $\lfloor i/s \rfloor$  and multiplying the low-resolution seam column by  $s$ . This mapped seam gives a center path on the original image. The implementation still computes the full-resolution energy map over the original image. The saving comes from the final full-resolution DP: instead of evaluating all columns, it is restricted to a narrow band around the mapped center path. With the default half-width 5, each row searches only from  $\text{center}[\text{row}] - 5$  to  $\text{center}[\text{row}] + 5$ , or about 11 columns, and the best seam inside this band becomes the final high-resolution seam. The `band_width` parameter therefore controls the speed-quality trade-off: a smaller band evaluates fewer full-resolution DP cells but constrains the seam more tightly to the low-resolution estimate, while a larger band is slower but allows the refined seam to deviate farther from the mapped center path.

The full DP cost for one seam is  $O(HW)$ . In the multi-scale version, the coarse DP costs approximately  $O(HW/s^2)$ , and the refinement DP costs  $O(HB)$ , where  $B$  is the full-resolution band width. Thus each seam costs about  $O(HW/s^2 + HB)$  instead of  $O(HW)$ . This can produce a much larger acceleration than batch extraction, but it is an approximate search: if the low-resolution seam is misleading or the refinement band is too narrow, the true global optimum may lie outside the searched region.

## 5.2 Extension Experiment Protocol and Results

The extension experiments use the same scalability protocol as Section 4: the source image is resized to six input resolutions from  $100 \times 67$  to  $512 \times 342$ , and each method reduces the width by half while keeping the height unchanged. To compare the extensions under one timing protocol, I run five methods on the same prepared inputs: backward energy, forward energy, batch seam extraction with batch size 5, local-DP update with  $\Delta = 5$ , and multi-scale seam carving with scale factor 2 and refinement band width 5. Each method-size pair is repeated five times, and Figure 3 plots the average runtime and shows the corresponding visual comparison.

The combined timing results match the theoretical expectations. Forward energy is the slowest method because it evaluates transition-dependent costs in addition to the DP recurrence. Batch seam extraction with batch size 5 stays close to the backward-energy baseline: at the largest size, backward energy averages 84.49 seconds and batch seam extraction averages 84.64 seconds. This is consistent with the complexity analysis, since the batch implementation still runs one full DP for each selected seam and only reduces the number of energy recomputations and deletion passes. Local-DP update is faster, averaging 59.80 seconds, because it reuses unaffected DP entries. Multi-scale seam carving is the fastest, averaging 27.57 seconds on the largest input, because it replaces most full-resolution DP work with a low-resolution seam search and narrow-band refinement.

Figure 3 also provides a qualitative check for the same largest benchmark image. The batch and local-DP variants are designed to remain close to the backward-energy baseline, while the multi-scale result may show a quality trade-off.



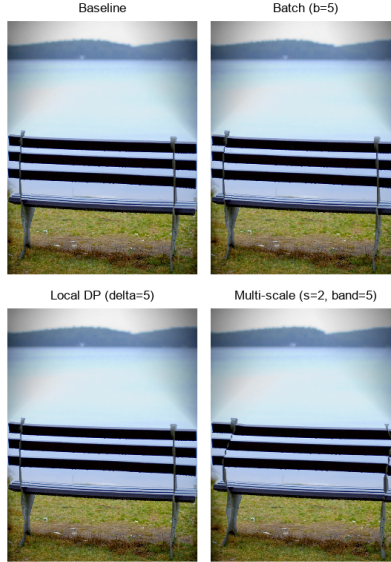
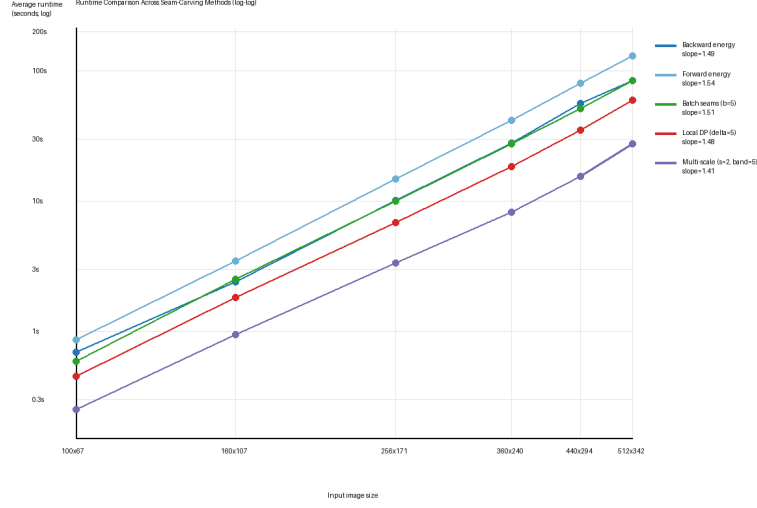


Figure 3: Extension experiment results. Top: unified runtime comparison for the two reproduced baselines and the three extension methods. All methods resize the same six inputs by halving the width while keeping the height unchanged. Bottom: visual comparison on the largest benchmark input, resized from  $512 \times 342$  to  $256 \times 342$  using the backward-energy baseline, batch seam extraction with batch size 5, local-DP update with  $\Delta = 5$ , and multi-scale seam carving with scale factor 2 and band width 5.

## References

- Shai Avidan and Ariel Shamir. Seam carving for content-aware image resizing. *ACM Trans. Graph.*, 26(3):10–es, July 2007. ISSN 0730-0301. doi: 10.1145/1276377.1276390. URL <https://doi.org/10.1145/1276377.1276390>.
- Michael Rubinstein, Ariel Shamir, and Shai Avidan. Improved seam carving for video retargeting. *ACM transactions on graphics (TOG)*, 27(3):1–9, 2008.
- Michael Rubinstein, Ariel Shamir, and Shai Avidan. Multi-operator media retargeting. *ACM Transactions on graphics (TOG)*, 28(3):1–11, 2009.

## A Reproduction Visual Results

This appendix collects the qualitative reproduction examples generated by the implementation. The panels illustrate vertical and horizontal seam removal, protective masks, seam insertion, object removal, and the qualitative difference between backward-energy and forward-energy seam carving.



Figure 4: Reproduction examples for reducing image dimensions by seam removal.



Figure 5: Reproduction examples for constrained resizing and content-aware enlargement.

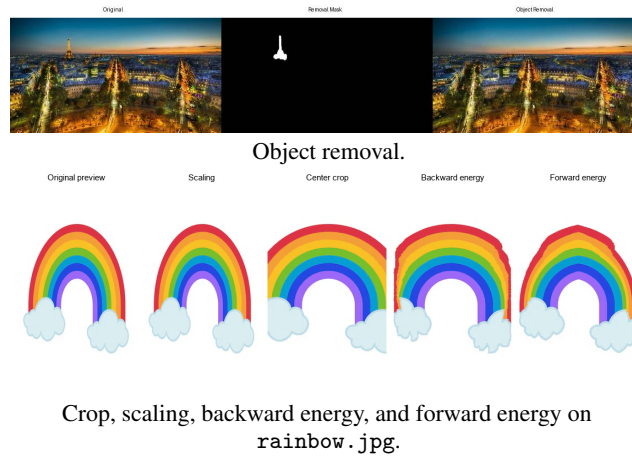


Figure 6: Reproduction examples for object removal and baseline/seam-carving comparison.